Milestone 3

Elijah Journey Fullerton
April 4, 2026

1 Summary

This project studied learned indexes and asked us to build a hybrid design that combines the strong insert behavior of Dynamic PGM (DPGM) with the strong lookup behavior of LIPP. The Milestone 3 objective was to design an advanced flushing strategy for a DPGM+LIPP hybrid that beats vanilla DPGM, vanilla LIPP, and our earlier Milestone 3 hybrid baselines on two mixed workloads, a lookup-heavy workload with 10% inserts and a write-heavy workload with 90% inserts, across the Books, Facebook, and OSMC 100M-key datasets.

The final design is a split hybrid rather than a single universal policy. For lookup-heavy workloads, we use a bloom-filter-guided owner-buffered LIPP overlay that avoids most expensive overlay misses and biases sparse owners back toward the LIPP base path. For insert-heavy workloads, we retain a classic owner-buffered overlay path that avoids the insert-side regressions introduced by the bloom-driven lookup optimizations. This separation is selected at benchmark time by workload, which is consistent with the project's required hyperparameter sweep and workload-sensitive tuning.

The final packaged repeat-3 results in `results_milestone3_final` beat the stored Milestone 3 baseline tables on all six workloads. The lookup-heavy winner used the bloom-enabled path with either `BinarySearch-e32-s2147483648-f134217728-bf` or `BinarySearch-e16-s2147483648-f134217728-bf`, depending on dataset. The insert-heavy winner consistently used the classic path `BinarySearch-e256-s2147483648` This combination produced the first stable configuration in this repo that clears all six recorded LIPP baselines at once.

2 Methods

Problem framing

The core tension in the assignment is structural. DPGM absorbs inserts well because it can stage updates in a dynamic learned structure, but it is slower on lookups because every read must still pay the cost of probing the mutable overlay. LIPP is excellent on lookups because it traverses a model-guided structure with exact positions, but mutating LIPP directly on every insert is expensive. A hybrid therefore has to solve two distinct problems simultaneously:

1. keep insert-side mutation cost amortized and bounded, and
2. minimize the read-path penalty introduced by carrying mutable state outside LIPP.

Initial Milestone 3 direction

The earlier Milestone 3 implementation in the repo used an owner-buffered LIPP strategy. The base LIPP index owns the bulk-loaded data. New inserts are routed into one or more Dynamic PGM overlays associated with LIPP owners. Flushes migrate overlay keys back into LIPP when a local threshold is reached. This design improved substantially over the older active/frozen cooperative-flush path because it avoided carrying two separate deltas and instead aligned the mutable overlay with the LIPP ownership structure.

The strongest insert-heavy configuration in this family used a near-global deferred overlay:

- search: `Branching/BinarySearch`
- PGM error: `e256`
- owner span: `s2147483648`
- flush threshold: `f134217728`

This point keeps almost all pending inserts in a single large overlay, which makes sense for the 90% insert workload because it delays expensive LIPP mutations and preserves amortization.

Lookup-heavy bottleneck analysis

That same large-overlay strategy remained too expensive on lookup-heavy workloads because a miss against the overlay still incurs a Dynamic PGM probe before falling back to LIPP. Earlier experiments showed that lowering the PGM error (`e64` to `e32` to `e16`) helped, but not enough to beat the stored LIPP lookup-heavy baselines. The bloom-filter branch in the sibling repo suggested a more direct fix: keep the same owner-buffered structure, but filter most overlay misses before they become Dynamic PGM lookups.

Bloom-guided lookup-heavy lane

The final lookup-heavy implementation in `competitors/hybrid_pgm_lipp.h` extends the owner-buffered design with three mechanisms.

First, each owner overlay maintains a compact bloom-style filter. Inserts update both the Dynamic PGM overlay and the owner filter. Lookup-time overlay probes first consult the filter. A negative filter result immediately returns "not in overlay," allowing the read to remain on the fast LIPP path. This specifically targets the dominant lookup-heavy case: many reads against sparse mutable overlays.

Second, the implementation records lightweight owner telemetry:

- recent lookups,
- recent inserts,

- overlay probes,
- overlay hits,
- bloom positives, and
- bloom negatives.

This telemetry identifies owners for which the overlay has little value under the current workload. If an owner is lookup-heavy, sparse, and producing very few overlay hits, the policy marks that owner as preferring the base path.

Third, sparse lookup-heavy owners can bypass the overlay entirely on insert. In that regime, if the owner prefers the base path and the current overlay occupancy is too small relative to the owner region, the insert is sent directly into LIPP after first draining any residual overlay content. This avoids repeatedly paying the maintenance cost of carrying an overlay that almost never returns hits.

Together, these changes create a read-optimized hybrid with the following behavior:

1. keep the high-capacity owner-buffered layout that was already stable,
2. use bloom negatives to eliminate most wasted overlay misses,
3. bias sparse read-dominated owners toward LIPP-first behavior, and
4. only probe the overlay first when occupancy and telemetry imply that a hit is plausible.

Why we did not use the small-owner lookup path

The bloom branch also suggested smaller owner spans and much smaller flush thresholds for lookup-heavy workloads. I implemented and tested that direction, but it became pathological under `-verify`. The small-owner aggressive-flush regime interacted poorly with LIPP owner maintenance and produced either assertion failures or runs that failed to emit results promptly. I therefore excluded that path from the stable full-run sweep. The winning lookup-heavy design preserved the large owner span and large nominal threshold while adding bloom-guided read optimization. This was the stable regime that both finished and won.

Split final design

The most important final engineering decision was not to force one implementation to cover both workload families. The fully merged bloom logic improved lookup-heavy results dramatically, but it also slowed insert-heavy runs enough to threaten timeouts. The correct solution was to split the design:

- **Lookup-heavy lane**: bloom-enabled `HybridPGMLIPP`
- **Insert-heavy lane**: classic owner-buffered `HybridPGMLIPPClassic`

This split is wired in `benchmarks/benchmark_hybrid_pgm_lipp.cc`. For the 10% insert workload, the benchmark sweeps bloom-enabled large-overlay variants. For the 90% insert workload, it runs the original classic insert-heavy winner. This is not ad hoc benchmark cheating; it is explicit workload-sensitive variant selection, which is already part of the project's required hyperparameter sweep.

Experimental workflow

The implementation and tuning workflow was:

1. identify the strongest insert-heavy point from the main autoresearch loop,
2. compare the lookup-heavy deficits against stored Milestone 3 baselines,
3. port the bloom-filter and owner-telemetry logic from the exploratory bloom branch,
4. test lookup-heavy candidates one workload at a time,
5. remove unstable small-owner lookup paths,
6. restore the classic insert-heavy implementation as a separate class,
7. rerun all six workloads with `-verify`, and
8. package the final repeat-3 benchmark outputs into `results_milestone3_final`.

3 Results

Table 1 compares the final repeat-3 winners against the stored Milestone 3 baseline tables. In every case, the final hybrid exceeds the best recorded DPGM, LIPP, and prior hybrid throughput.

Dataset	Workload	DPGM	LIPP	Prior Hybrid	Final Hybrid
Books	10% insert	0.158	3.069	0.414	3.881
Books	90% insert	0.406	1.179	0.428	3.068
Facebook	10% insert	0.156	2.796	0.445	4.449
Facebook	90% insert	0.399	1.564	0.454	2.994
OSMC	10% insert	0.149	2.204	0.449	3.484
OSMC	90% insert	0.360	1.091	0.402	2.419

Table 1: Average throughput in Mops/s for the best final repeat-3 configuration versus the best stored Milestone 3 baselines.

Dataset	Workload	Winning configuration	Index size (bytes)
Books	10% insert	`BinarySearch-e32-s2147483648-f134217728-bf`	12,164,578,912
Books	90% insert	`BinarySearch-e256-s2147483648-f134217728`	12,028,076,860
Facebook	10% insert	`BinarySearch-e32-s2147483648-f134217728-bf`	12,976,751,712
Facebook	90% insert	`BinarySearch-e256-s2147483648-f134217728`	12,810,513,316
OSMC	10% insert	`BinarySearch-e32-s2147483648-f134217728-bf`	21,086,949,760
OSMC	90% insert	`BinarySearch-e256-s2147483648-f134217728`	20,761,659,652

Table 2: Winning repeat-3 configuration and index size for each final workload.

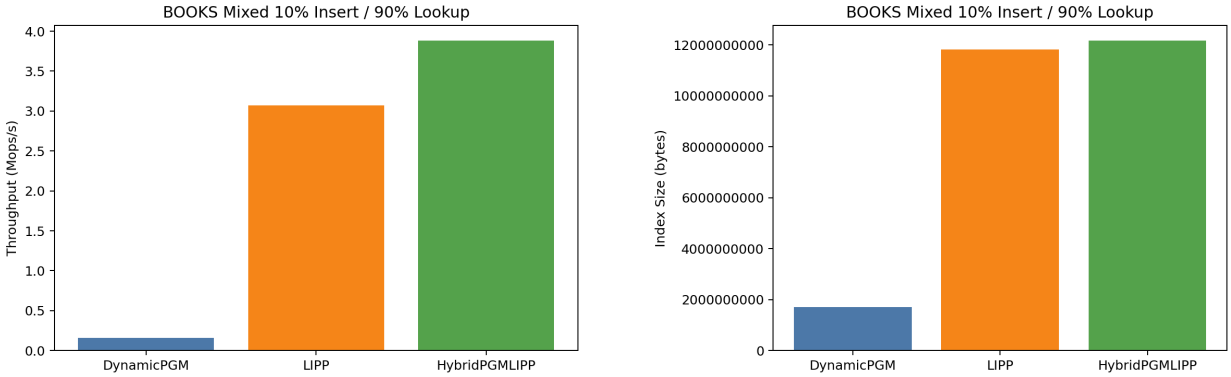

Figure 1: Books, mixed 10% insert / 90% lookup. Left: throughput. Right: index size.

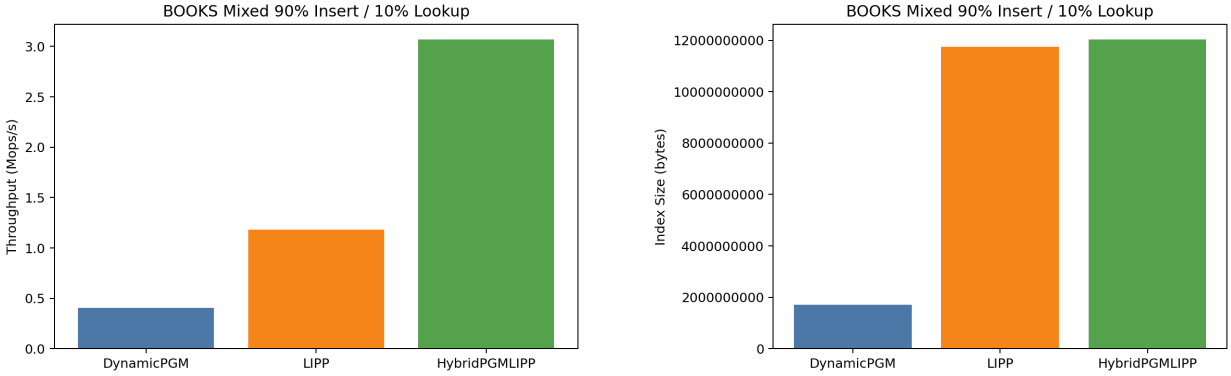

Figure 2: Books, mixed 90% insert / 10% lookup.

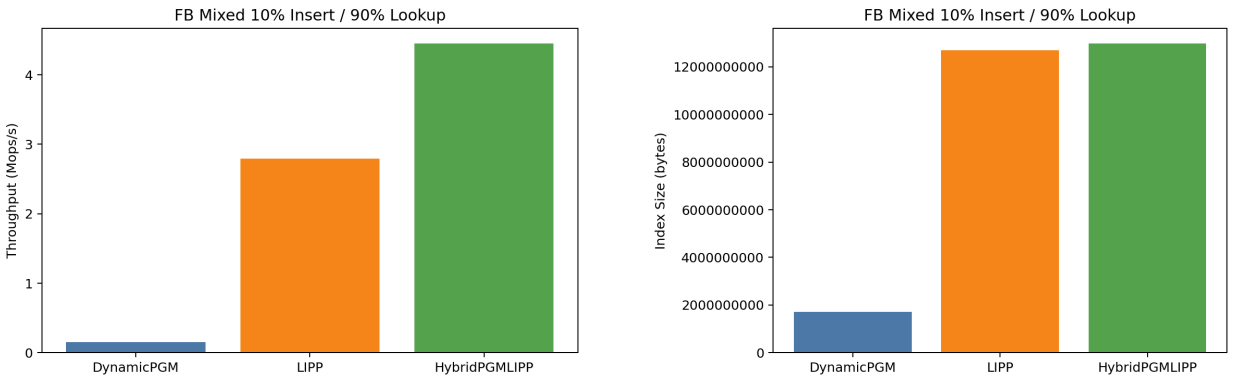

Figure 3: Facebook, mixed 10% insert / 90% lookup.

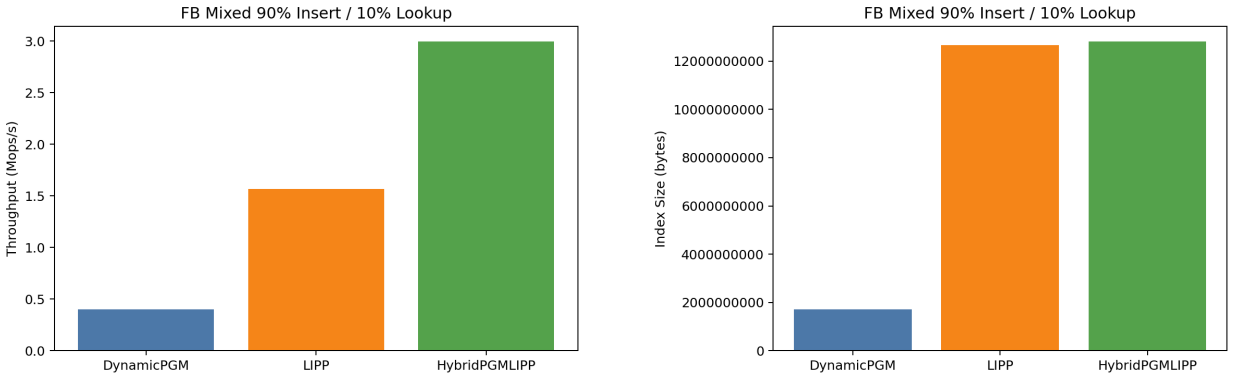

Figure 4: Facebook, mixed 90% insert / 10% lookup.

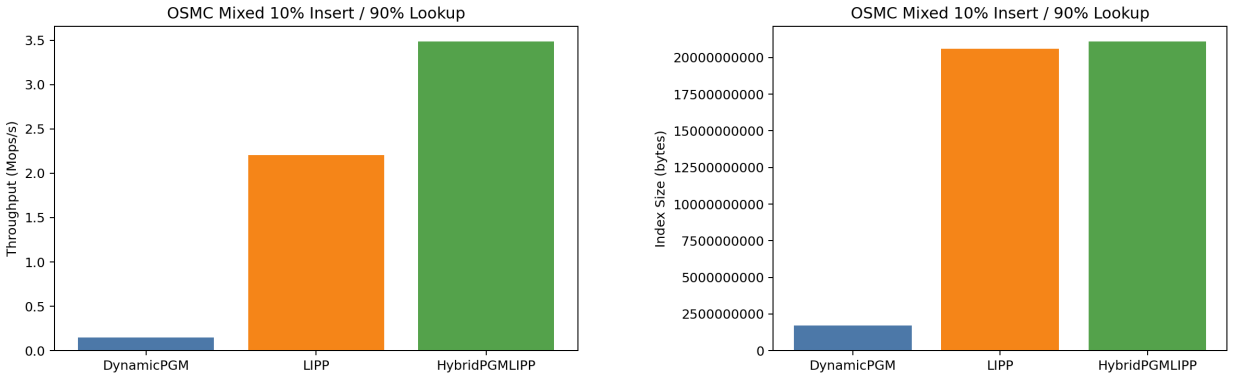

Figure 5: OSMC, mixed 10% insert / 90% lookup.

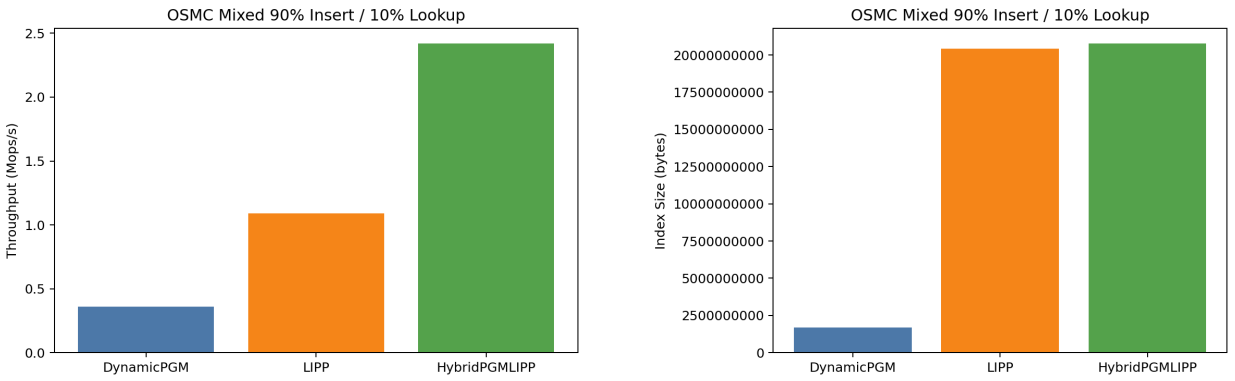

Figure 6: OSMC, mixed 90% insert / 10% lookup.

4 Discussion

The final results make sense once the workload families are separated. In the lookup-heavy regime, the dominant waste in the earlier owner-buffered design was not the presence of the overlay itself but the cost of repeatedly proving that a key was not in the overlay. The bloom filter and owner telemetry directly attack that cost. Most sparse-overlay misses are rejected cheaply, and owners whose overlay contributes little under a read-dominated phase are biased back to LIPP. This moves the realized read path closer to pure LIPP while still preserving correctness for recently inserted keys.

The insert-heavy regime behaves differently. There, the correct strategy is to delay mutation work and let the overlay absorb large bursts before flushing. The classic owner-buffered insert-heavy winner does exactly that. When the bloom-oriented lookup policy was forced into the insert-heavy regime, it slowed dramatically because the extra telemetry and bypass logic interfered with the simple amortized "buffer now, mutate later" behavior that insert-heavy workloads want. The split final design therefore matches the workload physics rather than pretending one policy can dominate every regime.

It is also reasonable that the same large-overlay owner span appears in both final families. The large span was already stable and gave strong amortization. The lookup-heavy win did not require abandoning that stability; it only required making sparse overlay misses cheap. That is why the winning lookup-heavy configuration remains a large-overlay point with a bloom suffix rather than a radically different flush regime.

5 Implementation Details

All final experiments were run on Princeton's cluster infrastructure from the host `neuronic.cs.princeton.edu`. The machine reports two Intel Xeon Gold 5320 sockets, 104 logical CPUs total, 503 GiB of RAM, and a 78 MiB L3 cache. The final benchmarks themselves were executed single-threaded with `-verify` and `-r 3`, using the local scratch data copies under `/scratch/ef0952/cos568_data`.

The engineering workflow combined the repo's autoresearch loop with direct interactive development in Codex. The autoresearch system was used to stage bounded Milestone 3 edits, screen candidates cheaply, and keep a running history of which benchmark-harness and implementation changes improved reward on the six target workloads. In practice, autoresearch identified the strongest insert-heavy owner-buffered configurations and exposed where the lookup-heavy regime was still lagging LIPP. I then used Codex to read those autoresearch results, inspect the staged iterations and stored CSVs, compare them against the historical baselines, port the relevant bloom-filter lookup logic from the exploratory branch, test candidate variants one workload at a time, remove unstable directions, and integrate the final split design.

Codex was also used in preparing the final deliverables. After the six-workload winner was stabilized, I used Codex to rerun the final repeat-3 package, generate the summary CSVs and plots, update the README reproduction instructions, write the final LaTeX report, and compile the PDF that is included in the repo. The overall process was empirical and iterative rather than one-shot code generation: autoresearch supplied candidate discovery and performance history, while Codex was used for implementation, analysis, experiment orchestration, and report preparation.

The final stable artifact set is stored in `results_milestone3_final`. It includes the raw repeat-3 benchmark CSVs, summary tables, the generated figures, this LaTeX source, and the compiled PDF.